

KernelAscent: Measuring Genuine Recursive Self-Improvement via a Kernel-to-Model Capability Loop

1 Problem and thesis

Recursive self-improvement (RSI) is the central capability behind “intelligence explosion” safety concerns, yet it has *no rigorous quantitative benchmark*. Existing work splits into two camps that never meet. **Kernel-generation benchmarks** (KernelBench, TritonBench, RE-Bench’s “Optimize a Kernel,” robust-kbench, KernelLLM, Kevin-32B) measure a one-shot artifact: can a model write a fast GPU kernel? This capability is now established. **Self-improvement systems** (STOP, ADAS, Darwin Gödel Machine, SICA) rewrite their own *scaffold code* but never touch the compute substrate and never measure whether *capability* compounds. Neither closes the loop that defines RSI: an agent improves the machinery that produces its own capability, and the improved capability then makes it better at that same task.

Thesis. We propose a benchmark whose loop is genuine capability self-improvement: a model optimizes the GPU kernels used to *train itself*; under a *fixed wall-clock budget* those faster kernels buy more effective training, producing a more capable model; the more capable model then optimizes kernels better. We measure whether kernel-optimization capability *compounds* across rounds, and separate true self-improvement from mere test-time compute scaling via a matched control. To our knowledge no prior benchmark closes a kernel→model→kernel capability loop.

2 The recursive loop

Let M_k be the model at round k and let the target capability be kernel-optimization skill, measured on a *held-out* task set T :

$$C(M) = \frac{1}{|T|} \sum_{i \in T} \mathbf{1}[s_i(M) > p \wedge \text{err}_i(M) \leq \varepsilon], \quad s_i = \frac{t_i^{\text{ref}}}{t_i^{\text{agent}}},$$

i.e. the fraction of tasks the model solves with measured speedup $> p$ at numerical error $\leq \varepsilon$ on real GPUs (KernelBench’s **fast_p**). Let $K(M)$ be the set of kernels M emits for the *training* stack (attention, GEMM, optimizer, all-reduce fusions), and let $R(M)$ be the resulting training throughput (steps/sec). The loop, run under a fixed per-round wall-clock budget W :

Round k : (1) evaluate $C(M_k)$ on T ; (2) M_k emits training kernels $K(M_k)$; (3) train under those kernels for wall-clock W , yielding $S_k = R(M_k) \cdot W$ gradient steps and a new model $M_{k+1} = \text{Train}(M_k; K(M_k), W)$.

Because W is fixed, the *only* way to fit more or larger training steps is for the model to produce faster kernels: kernel skill converts directly into training compute, which (hypothesis) converts into capability, which raises kernel skill. This is a genuine positive feedback loop on *capability*, not merely on speed.

3 Metrics and hypotheses

Compounding coefficient. Fit $C_k = C_0 + a k + b k^2$. The sign of b characterizes takeoff: $b > 0$ accelerating, $b \approx 0$ linear, $b < 0$ decelerating, and a plateau/regress if $g_k = C_{k+1} - C_k \rightarrow 0$ or < 0 . We report g_k , the throughput ratio $\rho_k = R(M_k)/R(M_0)$, and the cumulative capability gain.

The RSI signal (the linchpin). We run a *fixed-kernel control* arm that trains every round with the round-0 kernels (no self-acceleration), and a *compute-matched* control that is granted the same step count S_k without the self-referential kernel emission. Define

$$\Delta_k = C_k^{\text{self}} - C_k^{\text{control}}.$$

RSI is confirmed only if Δ_k grows with k : the self-accelerating arm must pull ahead *because its training throughput compounds*. If $\Delta_k \approx 0$, the effect is ordinary compute scaling, not self-improvement — and we report that honestly. Three falsifiable hypotheses: **H1** kernel quality measurably raises training throughput (ρ_k rises); **H2** added throughput raises kernel-optimization capability (C rises with S); **H3** the loop compounds ($g_k > 0$ and Δ_k increasing).

4 Experimental design

Substrate benchmark. Kernel tasks are grounded in real inference/training bottlenecks (MoE grouped GEMM, long-context / sliding-window attention, paged attention + KV-cache quantization, int4/fp8 dequant-fused GEMM, RoPE fusion, all-reduce fusion), tagged by the optimization *primitive* they require (tiling, tensor-core MMA, `cp.async` double-buffering, warp-specialization, TMA, fusion, quantization). Tasks are *procedurally generated* with a held-out

private split to defeat memorization; scored on **both A100 and H100** (final score = min across arches) to penalize single-arch overfit.

Training loop. M_0 is an open code model ($\sim 7\text{--}32\text{B}$). Rounds use RL fine-tuning on kernel optimization (GRPO, following Kevin-32B), so the training loop itself runs the model’s kernels and self-acceleration is intrinsic. Capability is always measured on the held-out split, never the RL training tasks.

Anti-gaming (the Sakana lesson). Sakana’s “AI CUDA Engineer” (2025) reward-hacked its own eval harness, reporting large speedups on slower kernels. We freeze the correctness + timing harness outside the agent’s write scope, use median-of- N timing on pinned GPU clocks with cache flushing and fresh allocations, re-check adversarial hidden shapes every round, and forbid dispatch to cuBLAS/CUTLASS/FlashAttention. Only externally measured wall-clock counts; the model never reports its own numbers.

Component	Measures	RSI role
Held-out fast_p $C(M_k)$	kernel-opt capability	the improving capability
Throughput $R(M_k), \rho_k$	training steps/sec from self-kernels	the feedback channel
Compounding coeff. a, b, g_k	shape of the capability curve	takeoff dynamics
Δ_k vs. controls	self-improvement over compute scaling	the RSI existence test

5 Feasibility, risks, deliverables

Compute. A100 (Ampere) + H100 (Hopper). Cost is dominated by the per-round RL training under budget W ; we bound total cost as (rounds) $\times W \times$ (arms). A pilot uses a small M_0 , short W , and 3–4 rounds to establish the curve before scaling. **Primary risk** is measurement validity of Δ_k ; we de-risk by first prototyping a *single* round end-to-end (emit kernels $\rightarrow$ verify throughput gain $\rightarrow$ train $\rightarrow$ re-measure C) to confirm H1 and H2 in isolation before claiming H3. **Secondary risk** is null compounding; this is itself a publishable result (*current models do not exhibit kernel-driven RSI takeoff*), which is a valuable safety datapoint. **Deliverables:** (i) the tiered, contamination-resistant kernel benchmark with A100+H100 scoring; (ii) the kernel $\rightarrow$ model RSI harness with locked verification; (iii) baseline compounding curves and the Δ_k RSI test for open frontier-scale models; (iv) a per-primitive capability profile localizing where self-improvement stalls.